Working Manual for This Neovim Configuration

Codex

2026-03-13

Contents

1 Purpose **2**

2 Executive Summary **2**

3 Mental Model of the Config **3**

4 Core Editing Defaults **3**
4.1 Leader Keys . . . 3
4.2 Numbers, Cursor, and Windows . . . 3
4.3 Search Behavior . . . 4
4.4 Indentation, Undo, and Text Display . . . 4
4.5 Clipboard and Mouse . . . 5
4.6 Diagnostics Display . . . 5

5 Daily Workflow **5**
5.1 Startup Experience . . . 5
5.2 Opening Files and Moving Around Projects . . . 5
5.3 Window Management . . . 6
5.4 Jumping Inside a Buffer . . . 6
5.5 Search, Lookup, and Discovery . . . 7
5.6 Terminal Workflow . . . 7
5.7 Writing and Commenting . . . 7

6 Language Workflows **8**
6.1 Lua . . . 8
6.2 Python . . . 8
6.3 Markdown . . . 9
6.4 Shell . . . 9
6.5 C++ and CUDA . . . 9

7 LSP, Completion, and Formatting **10**
7.1 LSP Model . . . 10
7.2 Completion . . . 10
7.3 Formatting . . . 10

8 Git and Project Awareness **11**
8.1 Gitsigns . . . 11
8.2 Todo Comments . . . 11

9 Treesitter and Text Objects **11**
9.1 Treesitter . . . 11

9.2 Mini Modules 12

10 Discoverability and Learning the Config **12**
10.1 Which-Key 12
10.2 Searching the Config Itself 12

11 Personal Usage Conventions This Config Encourages **13**

12 Limitations and Missing Pieces **13**

13 Troubleshooting **13**
13.1 Plugin Health and Setup 13
13.2 If Telescope Feels Incomplete 14
13.3 If LSP Is Missing in Python, Shell, or C++ 14
13.4 If Formatting Does Not Run 14
13.5 If System Clipboard Does Not Work 14

14 Command Appendix **14**
14.1 Core Commands 14
14.2 Search and Navigation Keymaps 14
14.3 Motion and Window Keymaps 15
14.4 Diagnostics and LSP Keymaps 15
14.5 Formatting and Environment Keymaps 15

15 Active Plugin Appendix **15**

16 Configuration Reading Guide **16**

17 Final Assessment **16**

1 Purpose

This manual explains the active Neovim configuration in this repository as it exists today. It is written for a user who already knows basic Vim motions and editing, but is still learning how this specific Lua configuration behaves.

The manual is hybrid by design:

- The first half is tutorial-oriented and explains how to work inside the editor day to day.
- The later sections are reference-oriented and collect commands, keymaps, and plugin behavior into lookup tables.

This document intentionally ignores plugin examples that exist in the repository but are not currently loaded.

2 Executive Summary

This configuration is a light customization of Kickstart Neovim. It is still compact and understandable from one main file, but it already provides a productive editing environment with:

- sensible editor defaults
- Telescope-based search and navigation
- Lua LSP support through Mason and `nvim-lspconfig`
- lightweight formatting through `conform.nvim`

- completion through `blink.cmp`
- Treesitter-based parsing
- Git signs in the gutter
- a dashboard, statusline, comments, snippets, Flash motion, ToggleTerm, and a Python virtual environment selector

The design philosophy is pragmatic rather than maximal:

- only a small set of tools are configured deeply
- the config is easy to read in `init.lua`
- the defaults support mixed work: Python, Markdown, shell, and some C++/CUDA editing
- some capabilities are present only partially, which is important to understand before relying on them

3 Mental Model of the Config

The most important file is `init.lua`. It does four jobs:

1. It sets core editor options.
2. It defines a small base layer of keymaps.
3. It bootstraps `lazy.nvim` if needed.
4. It declares and configures the active plugins.

The file `lua/custom/plugins/init.lua` is currently empty, so there are no extra custom plugin modules being added from there.

This matters because the config remains easy to inspect:

- if a behavior feels global, it is probably near the top of `init.lua`
- if a behavior feels plugin-specific, it is probably inside that plugin's `config`, `opts`, or `keys` block
- there is no deep module tree to chase through

Tradeoff:

- This is easier to learn than a large multi-file distribution.
- It becomes harder to maintain once you want per-language behavior, custom commands, or many more plugin integrations.

4 Core Editing Defaults

4.1 Leader Keys

The global leader and local leader are both set to space.

- `Leader`: `<Space>`
- `LocalLeader`: `<Space>`

This means many config-specific commands begin with `<leader>`.

4.2 Numbers, Cursor, and Windows

The editor uses:

- absolute line numbers
- relative line numbers
- `cursorline`

- splits opening to the right and below
- `scrolloff=10`

Practical effect:

- movement by relative counts is easier
- the current line is visually anchored
- opening splits usually preserves your sense of layout
- you keep context above and below the cursor

Tradeoff:

- relative numbers are efficient for movement but can feel visually noisy in prose writing
- `scrolloff=10` is comfortable for code reading but uses more vertical space

4.3 Search Behavior

Search uses:

- `ignorecase`
- `smartcase`
- `hlsearch`
- `inccommand=split`

This gives a good default search model:

- lowercase search is case-insensitive
- mixed-case search becomes case-sensitive
- substitutions preview in a split as you type them

The mapping `<Esc>` in normal mode clears search highlighting.

Tradeoff:

- this is ergonomic for daily use
- users expecting persistent search highlighting until the next search may be surprised by the clear-on-demand behavior

4.4 Indentation, Undo, and Text Display

The config enables:

- `breakindent`
- persistent undo via `undofile`
- visible list characters
- `virtualedit=block`

Visible list characters are configured as:

- tab: »
- trailing spaces: ·
- non-breaking spaces: ␣

Practical effect:

- wrapped lines preserve indentation visually
- undo history survives editor restarts
- whitespace mistakes are easier to detect
- visual block editing becomes more flexible

Tradeoff:

- visible whitespace is excellent for code and Markdown hygiene
- it can feel busy if you prefer a cleaner writing view

4.5 Clipboard and Mouse

The clipboard is set to `unnamedplus` on a scheduled startup call, which means Neovim tries to integrate with the system clipboard.

Mouse support is enabled for all modes with `mouse=a`.

Tradeoff:

- this makes copy/paste and terminal interoperability easier
- strict keyboard-only users may ignore mouse support entirely

4.6 Diagnostics Display

Diagnostics are configured with:

- virtual text enabled
- virtual lines disabled
- sorted by severity
- rounded floating windows
- underline starting at warning level
- no diagnostic updates while typing in insert mode

This is a conservative, readable setup. You see errors inline, but the display does not constantly churn while you are mid-insert.

Tradeoff:

- calmer editing experience
- slightly slower feedback while actively typing

5 Daily Workflow

5.1 Startup Experience

On startup, the dashboard plugin shows a Doom-style landing page with a custom ASCII header. This is mostly cosmetic, but it gives a clean entry point when no file is opened directly.

The colorscheme is `tokyonight-night` with italic comments disabled.

Tradeoff:

- visually polished and readable
- not heavily customized beyond the theme selection

5.2 Opening Files and Moving Around Projects

Your main project navigation tool is Telescope.

Use these as the primary discovery commands:

- `<leader>sf` for file search
- `<leader>sg` for live grep
- `<leader><leader>` for open buffers
- `<leader>s.` for recent files

- `<leader>sn` to search inside your Neovim config directory

Recommended daily habit:

1. Use `<leader>sf` to jump to files.
2. Use `<leader>sg` to search project-wide text.
3. Use `<leader><leader>` to move among working buffers.
4. Use `<leader>/` when you are already in the right buffer and only need local search.

Tradeoff:

- this is fast and consistent once memorized
- it assumes you are comfortable with fuzzy-finder workflows rather than a persistent file tree

5.3 Window Management

The config keeps window movement simple:

- `<C-h>` left
- `<C-j>` down
- `<C-k>` up
- `<C-l>` right

This encourages a split-based workflow without requiring custom window management plugins.

Recommended use:

- open searches or references in splits as needed
- navigate between code, terminal, and documentation panes with the same `Ctrl-hjkl` pattern

Tradeoff:

- minimal and dependable
- there are no custom resize or layout presets in the current active config

5.4 Jumping Inside a Buffer

The motion plugin `flash.nvim` is active.

Key behavior:

- `s` triggers Flash jump
- `S` triggers a Treesitter-aware jump
- `r` in operator-pending mode triggers remote flash
- `R` in operator-pending or visual mode triggers Treesitter search
- `<leader>j` is an additional jump entrypoint

Recommended use:

- use normal Vim motions for local edits
- use `s` or `<leader>j` when the target is visible but far enough away to make repeated motions inefficient

Tradeoff:

- faster screen-local navigation
- it changes traditional expectations for `s`, so you must consciously adopt the new motion

5.5 Search, Lookup, and Discovery

Telescope is the central lookup tool in this config. It covers:

- help tags
- keymaps
- files
- diagnostics
- commands
- recent files
- current word search
- current buffer fuzzy search
- open-file grep
- LSP references, definitions, implementations, type definitions, and symbols

Why this matters:

- one interface handles most interactive discovery
- you do not need separate mental models for file search, grep, and symbol lookup

Tradeoff:

- very efficient once internalized
- users who prefer plain quickfix-first workflows may find it more interactive than necessary

5.6 Terminal Workflow

`toggleterm.nvim` is installed with default settings. The current config does not define custom terminal keymaps beyond one important escape hatch:

- in terminal mode, `<Esc><Esc>` returns to normal mode

That means ToggleTerm is available, but not heavily opinionated in this config. You can still use its commands manually.

Recommended use for your workflow:

- run shell commands, Python tools, Markdown build commands, or CUDA/C++ compilation in a terminal pane
- use `<Esc><Esc>` to get back to window navigation quickly

Tradeoff:

- lightweight and unobtrusive
- not optimized with custom toggles, directions, or persistent named terminals

5.7 Writing and Commenting

`Comment.nvim` is active for comment toggling, using its default keymaps.

This is useful across:

- Python
- shell scripts
- Lua
- C++

Markdown does not use comments in the same way, but the rest of the writing ergonomics still matter:

- visible trailing spaces help avoid accidental Markdown formatting bugs
- break indentation improves wrapped text readability
- current buffer fuzzy search helps long-form navigation

Tradeoff:

- comment handling is easy with almost no config
- the manual assumes you will rely on plugin defaults rather than custom comment keymaps

6 Language Workflows

6.1 Lua

Lua is the only language with explicit LSP setup in the current config.

The config installs and enables:

- `lua_ls`
- `stylua`

Special handling is added so editing this Neovim config works well:

- Lua runtime is set to LuaJIT
- Neovim runtime files are added to the language server library
- third-party checks are reduced for this environment

This makes the config self-hosting: the editor is best configured to edit itself.

Tradeoff:

- excellent support for Neovim Lua
- other languages do not yet receive the same level of configuration

6.2 Python

Python support in this config is partial.

What is active:

- `venv-selector.nvim` can help you select a Python virtual environment with `,v`
- `pyright` is configured through Neovim LSP
- completion infrastructure exists through `blink.cmp`
- Treesitter-based syntax highlighting works when the Python parser is installed
- Telescope and terminal workflows support Python projects well

What is not explicitly configured:

- no Python formatter is wired into `conform.nvim`
- no Python linter is configured in the active config

Practical implication:

- you get real Python completion and LSP navigation through `pyright`
- you still need extra tooling such as `ruff`, `black`, or similar if you want linting or formatter integration

Tradeoff:

- the config now covers a solid Python editing baseline
- you still need to extend it if you want a fuller Python IDE-style workflow

6.3 Markdown

Markdown support is currently editor-centric rather than language-tool-centric.

What helps:

- visible whitespace
- Telescope search
- persistent undo
- comments are irrelevant here, but completion and snippets may still help in fenced code blocks or repetitive structures

What is not active:

- there is no active Markdown lint integration in the current config
- there is no dedicated Markdown preview plugin in the active config

Tradeoff:

- clean writing environment with strong navigation
- no built-in publishing or linting workflow beyond external tools you run manually

6.4 Shell

Shell editing benefits from:

- comment toggling
- Telescope search
- terminal integration
- Treesitter parsing if the parser is available

What is missing:

- no shell-specific formatter or linter is configured here

Tradeoff:

- good editing surface
- limited automation without further setup

6.5 C++ and CUDA

C++ editing is also partial.

What helps:

- Treesitter parsing
- completion framework
- Telescope navigation
- Git signs and diagnostics if an external LSP attaches by some other mechanism

Important formatting behavior:

- auto-format on save is explicitly disabled for `c` and `cpp`

This is a deliberate protective choice. C and C++ formatting can be destructive or surprising when no agreed formatter configuration is present.

Tradeoff:

- safer for heterogeneous C++ or CUDA codebases
- you must format manually or extend the config if you want automated formatting

7 LSP, Completion, and Formatting

7.1 LSP Model

The active LSP stack is:

- `nvim-lspconfig`
- `mason.nvim`
- `mason-lspconfig.nvim`
- `mason-tool-installer.nvim`
- `fidget.nvim`

LSP keymaps are attached only when a language server attaches to the current buffer.

Active LSP mappings:

- `grn` rename symbol
- `gra` code action
- `grD` declaration
- `grd` definition
- `gri` implementation
- `grr` references
- `grt` type definition
- `gO` document symbols
- `gW` workspace symbols
- `K` hover
- `<leader>th` toggle inlay hints if supported

Tradeoff:

- buffer-local mappings avoid polluting unrelated buffers
- you only get the full experience on languages that actually have an attached server

7.2 Completion

Completion uses `blink.cmp` with:

- default keymap preset
- LSP source
- filesystem path source
- snippet source through LuaSnip
- signature help enabled
- completion documentation disabled by default unless requested

This keeps completion useful without becoming visually noisy.

Tradeoff:

- lighter experience, especially for focused editing
- less always-on documentation than some IDE-like completion UIs

7.3 Formatting

Formatting is handled by `conform.nvim`.

Manual formatting:

- `<leader>f`

Automatic formatting on save:

- enabled in general
- disabled for `c` and `cpp`

Current formatter mappings by filetype:

- `lua -> stylua`

Fallback behavior:

- if a formatter is not configured, Conform can fall back to LSP formatting where possible

Important implication:

- formatting is robust for Lua
- other languages depend on external formatter setup or LSP support that is not yet declared here

Tradeoff:

- safe and minimal
- not yet a full multi-language formatting pipeline

8 Git and Project Awareness

8.1 Gitsigns

`gitsigns.nvim` is active with custom sign characters in the sign column:

- added lines: `+`
- changed lines: `~`
- deleted lines: `_`
- top delete: `‾`
- changed delete: `~`

No extra Git keymaps are currently active in the loaded config. The plugin still provides visual context in the gutter.

Tradeoff:

- useful signal with almost no complexity
- less powerful than a full hunk-management setup with custom mappings

8.2 Todo Comments

`todo-comments.nvim` is active. It highlights TODO-style annotations in comments, but sign-column icons are disabled.

Tradeoff:

- useful for scanning work markers in code
- intentionally understated in the UI

9 Treesitter and Text Objects

9.1 Treesitter

`nvim-treesitter` is enabled eagerly and starts parsing on `FileType` when a parser can be loaded for that language.

It also sets indentation via Treesitter.

Practical effect:

- better syntax awareness
- improved structural motions and plugin integrations when supported
- cleaner highlighting across many filetypes

Tradeoff:

- stronger syntax model than regex highlighting
- parser availability still matters language by language

9.2 Mini Modules

The active `mini.nvim` modules are:

- `mini.ai`
- `mini.surround`
- `mini.statusline`

What this means in practice:

- text object selection is improved
- surround editing is available
- the statusline stays lightweight

Tradeoff:

- small, composable features with low maintenance cost
- less discoverable than more opinionated, highly themed plugins

10 Discoverability and Learning the Config

10.1 Which-Key

`which-key.nvim` is configured with zero delay, so leader-based mappings are exposed immediately.

This is especially useful while learning the config because it lowers the memory burden of:

- search commands under `<leader>s`
- toggle commands under `<leader>t`
- future code-related groups

Tradeoff:

- excellent for learning and recall
- can slightly encourage menu-driven habits instead of pure memorization

10.2 Searching the Config Itself

Use:

- `<leader>sn` to search files inside the Neovim config directory
- `<leader>sk` to search known keymaps
- `<leader>sh` to search help tags
- `<leader>ss` to browse Telescope pickers

This is the fastest way to study the config while using it.

11 Personal Usage Conventions This Config Encourages

Because you asked to include all practical conventions, this section documents the conventions implied by the active setup, even when they are not hard-enforced.

Recommended conventions:

- Use Telescope first for files, grep, buffers, and config discovery.
- Use splits as the main workspace layout tool and move with `Ctrl-hjkl`.
- Use Flash for visible-target jumps instead of repeating single-character motions many times.
- Use `<Esc>` to clear search highlight instead of leaving the screen noisy.
- Use `<leader>e` and `<leader>q` as the first diagnostic triage tools.
- Use `<leader>f` as the standard formatting entrypoint.
- Use `,v` when working in Python projects that depend on a virtual environment.
- Use ToggleTerm for shell-heavy work rather than leaving the editor for every command.
- Use relative numbers as a movement aid, not as visual clutter to be ignored.
- Use `<leader>sn` and `lua_ls` support to treat the config itself as a project you can iteratively improve.

12 Limitations and Missing Pieces

It is important to understand what this config does not yet do.

Currently missing or partial:

- no active file tree plugin
- no active autopairs plugin
- no active lint plugin
- no Python formatter integration
- no shell formatter integration
- no Markdown-specific preview or linting integration
- no active Git hunk keymap layer beyond signs
- no active debugger setup
- no explicit C/C++ LSP configuration

This does not make the config weak. It makes it focused. But for Python, Markdown publishing, or CUDA-heavy development, you will eventually want targeted additions.

13 Troubleshooting

13.1 Plugin Health and Setup

Useful commands:

- `:Lazy`
- `:Mason`
- `:checkhealth`
- `:ConformInfo`

Use these when:

- a plugin appears missing
- a language server is not attaching
- formatting is not happening
- clipboard, Treesitter, or provider support feels broken

13.2 If Telescope Feels Incomplete

Check whether external tools such as `ripgrep` and `make` are installed. Telescope still works without every optional enhancement, but some speedups or native extensions depend on those tools.

13.3 If LSP Is Missing in Python, Shell, or C++

Shell and C/C++ still have no explicit LSP configuration here. Python should attach through `pyright`, so if it does not, check `:LspInfo`, `:Mason`, and whether `pyright` is installed and available.

13.4 If Formatting Does Not Run

Check:

- whether the current filetype has a formatter configured
- whether the language server supports formatting fallback
- whether you are editing `c` or `cpp`, where format-on-save is intentionally disabled

13.5 If System Clipboard Does Not Work

Run `:checkhealth` and verify your platform clipboard provider is installed and detected.

14 Command Appendix

14.1 Core Commands

Command	Purpose
`:Lazy`	Inspect plugin installation and status
`:Mason`	Inspect and manage external tools and LSP packages
`:checkhealth`	Run Neovim health diagnostics
`:ConformInfo`	Inspect formatting setup
`:VenvSelect`	Choose a Python virtual environment

14.2 Search and Navigation Keymaps

Key	Purpose
`<leader>sh`	Search help tags
`<leader>sk`	Search keymaps
`<leader>sf`	Search files
`<leader>ss`	Search Telescope pickers
`<leader>sw`	Search current word
`<leader>sg`	Live grep
`<leader>sd`	Search diagnostics
`<leader>sr`	Resume previous Telescope picker
`<leader>s.`	Search recent files
`<leader>sc`	Search commands
`<leader><leader>`	Search open buffers
`<leader>/`	Fuzzy search current buffer

Key	Purpose
`<leader>s/`	Grep in open files
`<leader>sn`	Search Neovim config files

14.3 Motion and Window Keymaps

Key	Purpose
`<C-h>`	Move to left window
`<C-j>`	Move to lower window
`<C-k>`	Move to upper window
`<C-l>`	Move to right window
`s`	Flash jump
`S`	Flash Treesitter jump
`r`	Remote flash in operator-pending mode
`R`	Treesitter search in operator-pending or visual mode
`<leader>j`	Flash jump
`<Esc><Esc>`	Exit terminal mode to normal mode
`<Esc>`	Clear search highlighting in normal mode

14.4 Diagnostics and LSP Keymaps

Key	Purpose
`[d`	Previous diagnostic
`]d`	Next diagnostic
`<leader>e`	Floating diagnostic details
`<leader>q`	Diagnostics to location list
`grn`	Rename symbol
`gra`	Code action
`grD`	Declaration
`grd`	Definition
`gri`	Implementation
`grr`	References
`grt`	Type definition
`gO`	Document symbols
`gW`	Workspace symbols
`K`	Hover documentation
`<leader>th`	Toggle inlay hints when supported

14.5 Formatting and Environment Keymaps

Key	Purpose
`<leader>f`	Format current buffer
`,v`	Open Python virtual environment selector

15 Active Plugin Appendix

The following plugins are active in the current config:

Plugin	Role
`folke/lazy.nvim`	Plugin manager bootstrap
`NMAC427/guess-indent.nvim`	Detect indentation settings
`numToStr/Comment.nvim`	Comment toggling
`akinsho/toggleterm.nvim`	Embedded terminal management
`linux-cultist/venv-selector.nvim`	Python virtualenv selection
`folke/flash.nvim`	Fast jump motion
`shortcuts/no-neck-pain.nvim`	Centered writing/layout plugin, loaded with defaults
`nvimdev/dashboard-nvim`	Startup dashboard
`lewis6991/gitsigns.nvim`	Git signs in gutter
`folke/which-key.nvim`	Keymap discovery
`nvim-telescope/telescope.nvim`	Fuzzy finder and search hub
`nvim-telescope/telescope-fzf-native.nvim`	Native Telescope sorter when `make` is available
`nvim-telescope/telescope-ui-select.nvim`	Telescope-backed UI select menus
`neovim/nvim-lspconfig`	LSP client configuration
`mason-org/mason.nvim`	External tool management
`mason-org/mason-lspconfig.nvim`	Mason bridge for LSP
`WhoIsSethDaniel/mason-tool-installer.nvim`	Ensures requested tools are installed
`j-hui/fidget.nvim`	LSP progress UI
`stevearc/conform.nvim`	Formatting
`saghen/blink.cmp`	Completion
`L3MON4D3/LuaSnip`	Snippet engine
`folke/tokyonight.nvim`	Colorscheme
`folke/todo-comments.nvim`	TODO comment highlighting
`nvim-mini/mini.nvim`	`mini.ai`, `mini.surround`, `mini.statusline`
`nvim-treesitter/nvim-treesitter`	Parser-based highlighting and indentation

16 Configuration Reading Guide

If you want to learn the Lua config itself, read it in this order:

1. `init.lua` lines 1-54 for global options, diagnostics, and keymaps.
2. `init.lua` lines 67-180 for plugin declarations and lightweight plugins.
3. `init.lua` lines 182-243 for Telescope behavior.
4. `init.lua` lines 246-341 for LSP and Mason.
5. `init.lua` lines 343-450 for formatting, completion, theme, mini modules, and Treesitter.

This order mirrors how the user experiences the config:

- editor defaults first
- navigation next
- language features after that
- implementation details last

17 Final Assessment

This is a solid, learnable Neovim configuration for mixed use. Its strongest area is still editing Neovim Lua itself, plus general-purpose project navigation through Telescope. Python now has a stronger baseline because LSP completion and Treesitter highlighting are in place,

but language-specific tooling outside Lua remains intentionally light overall, especially for formatting, linting, and C++/CUDA workflows.

That is not a flaw in itself. It is the natural result of a compact config that prioritizes clarity over feature sprawl. The main value of this setup is that you can understand it, use it immediately, and extend it deliberately.